Day 9 — Chunking + Embedding + 검색(RAG 완성)

> ① **Chunking** — 문서 → 깨끗한 조각 8개 ✅ ② **Embedding** — 조각 1개 → 숫자 3072개(벡터) ✅ ③ **8개 전부 임베딩** → `vectors.json` **저장** ✅ ④ **검색** — 질문과 제일 가까운 조각 찾기 ✅ 파일: `Kids_Feedback/rag/kids.py`(①②③) · `Kids_Feedback/rag/search.py`(④)
>
> 👉 **전체 그림이 궁금하면 맨 아래 [21. 전체 흐름] 먼저 읽어도 됨.**

🧰 0. 한눈에 보기 — RAG 4조각 함수 & 흐름 사전

> 복습용 빠른 참고. **자세한 설명은 각 섹션 번호로.**

데이터가 변해가는 흐름 (모양이 어떻게 바뀌나)

```
rag_docs/위축형_칭찬법.md   (문서 1개, 글자 덩어리)
   │ ① 자르기
   ▼
chunks   = ["질문:…", "질문:…", …]        (문자열 8개 리스트)
   │ ② 숫자로
   ▼
vectors  = [[0.1, …3072개], …]            (숫자 리스트 8개)
   │ ③ 짝지어 저장
   ▼
vectors.json = [{"text":…, "vector":[…]}, …8개]   (파일)
   │ ④ 질문과 비교
   ▼
best 조각 1개   ("질문과 뜻이 제일 가까운 그거")
```

조각별 "내가 쓴 함수·도구" 사전

① Chunking (자르기) — `kids.py` · 섹션 1~4

도구	하는 일
`open(경로, "r", encoding="utf-8")` + `.read()`	문서 파일을 열어서 글자 전체 읽기
`문자열.split("##")`	`##` 기준으로 자르기 (기준 글자는 버려짐)
`리스트[1:]`	0번(머리말) 버리고 1번부터
`문자열.strip()`	앞뒤 공백·줄바꿈 털기 (문자열에만, 리스트엔 ❌)
`빈통=[]` → `for` → `.append()`	하나씩 꺼내 가공해서 담기 (핵심 뼈대)

② Embedding (글→숫자) — `kids.py` · 섹션 7~12

도구	하는 일
`load_dotenv("Kids_Feedback/.env")`	`.env`에서 API 키 불러올 준비
`os.getenv("GEMINI_API_KEY")`	키 값 꺼내기
`genai.Client(api_key=…)`	제미나이 점원 부르기 (프로그램당 1번)
`embed_feedback(글)` ← 내가 만든 함수	글 1개 → 숫자 3072개. 안에서 `client.models.embed_content(...)` 호출
`response.embeddings[0].values`	응답 상자 3겹 벗겨 진짜 숫자 목록 꺼내기
`len()` / `리스트[:5]`	개수 세기(3072 확인) / 앞 5개만 맛보기

③ 저장 (한 번 만들고 재사용) — `kids.py` · 섹션 16, 21

도구	하는 일
`빈통=[]` → `for` → `.append()`	조각 8개 전부 임베딩해서 `vectors`에 쌓기
`zip(chunks, vectors)`	글과 벡터를 **짝지어** 하나씩 꺼내기
`.append({"text": c, "vector": v})`	`{글, 숫자}` 딕셔너리로 묶기
`json.dump(data, f, ensure_ascii=False)`	파일로 저장 (`ensure_ascii=False` = 한글 안 깨지게)
💡 왜 저장?	임베딩은 API 호출(돈·시간). 한 번 만들면 다음엔 **읽기만**

④ 검색 (질문과 제일 가까운 것) — `search.py` · 섹션 18~20

도구	하는 일
`json.load(f)`	저장해둔 `vectors.json` 다시 읽기
`embed_feedback(question)`	질문도 **같은 함수**로 숫자화 (for 밖, 1번만)
`cos_sim(a, b)` ← 내가 만든 함수	두 벡터 닮음 점수 (1=닮음, 0=남남). 내적÷(크기·크기)
`math.sqrt(...)`	벡터 크기 구할 때 제곱근
`빈통=[]` → `for d in data` → `.append()`	8개 각각과 비교해 점수 쌓기
`scores.index(max(scores))`	제일 높은 점수의 **자리 번호** (`data`와 순서 같아서 그 번호 조각이 정답)

내가 직접 만든 함수 2개 (꼭 기억)

- **`embed_feedback(글)`** → 글을 숫자 3072개로. ②에서 만들고 ④에서 **재사용**(질문 숫자화).
- **`cos_sim(a, b)`** → 두 숫자목록이 얼마나 닮았나 점수. ④에서 8번 비교에 씀.

아직 안 만든 것 (Day 10)

- 🟡 **3단계 생성** — 찾은 조각을 프롬프트에 끼워 `generate_feedback(prompt)` 호출 → 최종 답.
  (`generate_feedback`는 Day 4에 만든 것, `llm_service.py`에 있음 → 재사용)

---

① Chunking (조각 자르기)

---

1. 오늘 완성한 코드

```
with open("rag_docs/위축형_칭찬법.md", "r", encoding="utf-8") as f:
    text = f.read()
chunking = text.split("##")

chunks = []
for c in chunking[1:]:
    chunks.append(c.strip())

print(len(chunks))
print(chunks[0])
```

결과

```
8
질문: 위축형 아이가 계속 실패하던 걸 드디어 해냈을 때 뭐라고 칭찬하나요?
    (초안) 결과가 아니라 "포기하지 않고 다시 해본 시도"를 콕 집어 칭찬한다...
```

---

2. 핵심 개념

chunking = 문서를 조각으로 자르기

RAG의 첫 단계. 문서를 통째로 쓰면 안 되고 **질문 1개 = 대답 1개** 단위로 잘라야 검색이 된다. (조각 기준은 Day 8에서 세움 → `study/day8.md`)

우리 문서는 `##`로 질문을 구분해놨으니까, `##` **기준으로 자르면 = 질문 단위로 잘린다.**

`.split("##")` — 기준 글자로 자르기

```
"A##B##C".split("##")   # ['A', 'B', 'C']
```

⚠️ `split`**은 자르는 기준이 된 글자를 버린다.** 그래서 `##  질문:` 이 `질문:` 으로 남는다 (앞에 공백 한 칸 붙은 채로).

왜 `[1:]` 로 0번을 버리나

`split` 결과의 **0번은 첫 `##` 앞에 있던 내용** = 문서 머리말이다.

```
[
  '# 위축형 아이 — 칭찬법\n\n> 조각 = 질문 1개...',   ← 0번: 머리말 (쓰레기)
  ' 질문: 계속 실패하던 걸 드디어...',                ← 1번부터 진짜 조각
  ' 질문: 강사가 시킨 심부름을...',
]
```

머리말은 질문-대답이 아니라서 검색에 쓰면 방해만 된다 → **버린다.**

슬라이싱 `[1:]`

```
과일 = ["쓰레기", "사과", "바나나"]
과일[0]      # '쓰레기'          ← 0번 하나
과일[1:]     # ['사과','바나나']  ← 1번부터 끝까지
```

- 파이썬은 **0번부터** 센다
- 끝 숫자를 안 쓰면 "끝까지"

`.strip()` — 앞뒤 공백/줄바꿈 털기

```
"  안녕\n".strip()    # '안녕'
```

- **앞뒤만** 지운다. 가운데 공백은 안 건드림
- **문자열 하나**에만 쓸 수 있음 → 리스트엔 못 씀
  - `chunking.strip()` ❌ `AttributeError: 'list' object has no attribute 'strip'`
  - 그래서 `for`로 하나씩 꺼내서 각각 strip 해야 함

---

3. for문 기본 뼈대 (제일 많이 쓰는 형태)

> **빈 통 만들기 → `for`로 하나씩 꺼내기 → 가공해서 `append` → 반복 밖에서 `print`**

```
chunks = []                          # ① 빈 통 (먼저 안 만들면 NameError)
for c in chunking[1:]:               # ② 하나씩 꺼내서 c에 담기, 끝에 콜론(:) 필수
    chunks.append(c.strip())         # ③ 가공해서 통에 넣기 (들여쓰기 = 반복 안)
print(chunks)                        # ④ 반복 끝나고 한 번만 (들여쓰기 없음!)
```

- `c`는 내가 지은 **이름표**일 뿐. `chunk`든 `x`든 아무거나 됨
- `chunks.append(c.strip())` 은 **안에서 바깥으로** 읽는다
  1. `c.strip()` 먼저 실행 → 깨끗해진 문자열
  2. `append`가 그걸 받아서 통에 넣음
- **들여쓰기가 문법이다.** 들여쓰면 "매 바퀴 실행", 안 들여쓰면 "다 끝나고 한 번"

🐞 **오늘 실수:** `print(chunks)`를 들여써서 반복 안에 넣음 → 8번 찍힘 (통에 하나씩 쌓이는 과정이 다 보임. 틀린 건 아니고 중간과정 볼 땐 일부러 이렇게도 함)

📝 연습문제: `study/연습_for문.md`

---

4. 결과 확인법 — 조각 하나만 찍어보기

`print(chunks)` 로 리스트 전체를 찍으면 한 덩어리로 쏟아져서 눈으로 못 본다. **`chunks[0]` 하나만** 찍으면 두 가지가 동시에 확인된다.

`chunks[0]` 이 뭘로 시작하나	판정
`질문:` 바로 시작	✅ 통과
`# 위축형 아이 – 칭찬법`	❌ 머리말 안 지워짐 → `[1:]` 확인
`질문:` (앞 공백 한 칸)	❌ strip 안 먹음 → `.strip()` 확인

왜 하나만 봐도 되나: 0번 자리는 원래 머리말 자리였다. 거기에 `질문:`이 앉아 있으면 = 머리말이 밀려났다는 증거.

`len(chunks)` = 조각 개수. 문서에 `##` 질문이 8개니까 **8이 정답.** 9가 나오면 머리말이 아직 껴 있는 것.

---

5. 🐞 오늘 만난 에러 3개 (전부 코드 문제가 아니었음)

① FileNotFoundError — 상대경로

```
FileNotFoundError: No such file or directory: 'rag_docs/위축형_칭찬법.md'
```

`"rag_docs/..."` 같은 상대경로는 "명령어를 친 위치" 기준으로 찾는다. 파이썬 파일이 있는 위치가 아니다. ← 여기서 다 걸린다.

② 폴더 이름이 두 개 겹침 (오늘의 최대 함정)

```
C:/Users/USER/GITHUB/
└── Kids_Feedback/          ← ① 바깥. rag_docs가 여기. 여기서 실행해야 됨
    ├── rag_docs/
    ├── study/
    └── Kids_Feedback/      ← ② 안쪽. 이름이 똑같음! 여기선 안 됨
        └── rag/kids.py
```

`rag/`에서 상위로 올라가면 ②에 도착 → 이름이 같아서 맞게 온 줄 알지만 `rag_docs`는 ①에 있음.

제대로 왔는지 1초 확인: `ls` 쳐서 `rag_docs`, `study`, `todayplan.md`가 보이면 맞음.

③ UnicodeEncodeError — 인코딩

```
UnicodeEncodeError: 'cp949' codec can't encode character '–'
```

- **인코딩** = 글자를 숫자로 바꾸는 **규칙표**
- **UTF-8** = 요즘 표준 (한글·이모지·– 다 됨) / **cp949** = 윈도우 옛날 한국어 표 (– 없음)
- 파일은 `encoding="utf-8"`로 잘 읽었는데, **화면에 찍을 때** 윈도우가 cp949로 바꾸려다 –에서 터짐 (문서 머리말 `# 위축형 아이 – 칭찬법` 의 긴 대시 하나 때문)

---

6. 실행 명령어 (PowerShell)

```
cd C:\Users\USER\GITHUB\Kids_Feedback
$env:PYTHONIOENCODING="utf-8"
python Kids_Feedback\rag\kids.py
```

명령어에 위치가 두 개 들어간다:

- `cd ...` → **어디에 서서** 실행할까
- `python Kids_Feedback\rag\kids.py` → **어느 파일을** 실행할까 (서 있는 곳부터 파일까지 가는 길 전부)

경로에 `Kids_Feedback`이 두 번 나오는 게 맞다. 오타 아님.

`$env:PYTHONIOENCODING="utf-8"` 이 뭐냐

- 파이썬한테 "찍을 때도 cp949 말고 **UTF-8 써**"라고 알려주는 것
- `$env:이름="값"` = PowerShell **환경변수** 설정 문법. 환경변수 = 프로그램이 참고하는 **설정 쪽지**. 코드를 안 고치고 바깥에서 동작을 바꿀 때 씀
- **터미널 창 하나당 한 번만.** 창 닫으면 사라져서 새 창에선 다시 쳐야 함

⚠️ `PYTHONIOENCODING=utf-8 python ...` 처럼 한 줄로 붙여 쓰는 건 **Mac/Linux(bash) 문법**이라 PowerShell에선 안 먹는다.

---

② Embedding (조각 → 숫자)

> 결과: 조각 1개 → **숫자 3072개** ✅

---

7. 임베딩이 뭐고 왜 하나

임베딩 = 글을 숫자 목록으로 바꾸는 것.

컴퓨터는 "위축된 아이 칭찬"과 "소극적인 애 격려"가 비슷하다는 걸 **글자로는** 못 따진다. 겹치는 글자가 하나도 없으니까. 그래서 각각을 숫자 목록으로 바꿔놓고 **계산으로** 비교한다.

좌표로 생각하면 쉽다:

```
사과   = (3, 5)
배     = (3.1, 4.9)   ← 가까움  = 비슷한 뜻
자동차 = (-20, 88)    ← 멂      = 다른 뜻
```

칸이 2개면 뜻을 2가지 기준으로만 구분한다. **3072칸이면 3072가지 기준**으로 미세하게 구분한다.

---

8. 완성한 코드

```python
import os
from dotenv import load_dotenv
from google import genai

load_dotenv("Kids_Feedback/.env")
api_key = os.getenv("GEMINI_API_KEY")

client = genai.Client(api_key=api_key)

def embed_feedback(prompt):
    response = client.models.embed_content(
        model="gemini-embedding-001",
        contents=prompt
    )
    return response.embeddings[0].values

# --- (중간: chunking 코드) ---

vector = embed_feedback(chunks[0])
print(len(vector))
print(vector[:5])
```

결과

```
8
질문: 위축형 아이가 계속 실패하던 걸 드디어 해냈을 때...
3072
[-0.01434503, 0.010975923, 0.013552408, -0.051176287, 0.0069616917]
```

---

9. 왜 3072인가 — 벡터의 길이

`gemini-embedding-001` 모델이 정한 **칸 수**. 어떤 글을 넣든 **항상 3072개**로 나온다.

```
"위축형 아이 칭찬법..." (긴 글)  → 숫자 3072개
"안녕"                  (짧은 글) → 숫자 3072개
```

글 길이와 무관하게 고정. 왜? 👉 **길이가 같아야 비교(계산)가 되기 때문.** 조각마다 길이가 제각각이면(3072개 vs 500개) 좌표 비교 자체가 불가능하다.

---

10. 응답 상자 3겹 벗기기

`embed_content(...)`가 돌려주는 건 숫자 리스트가 **아니라 포장 상자**다.

```
response
 └ .embeddings      ← 리스트 (여러 개 한 번에 넣을 수도 있어서)
    └ [0]           ← 그중 첫 번째
       └ .values    ← 여기가 진짜 숫자 목록
```

그래서 `return response.embeddings[0].values` — 상자를 3겹 벗기는 코드다.

---

11. 🔴 오늘의 최대 난관 — `def`(만들기) vs 호출(부르기)

여기서 제일 많이 막혔다. **반드시 다시 볼 것.**

`def`는 "만들기"지 "부르기"가 아니다

```
def 라면끓이기(재료):      # ← 레시피를 종이에 적는 것. 아직 아무도 안 먹음
    ...

라면끓이기(계란)           # ← 실제로 끓이는 것. 여기서 처음 일이 벌어짐
```

`def`가 붙어 있으면 **무조건 "만들기"**다. 몇 번을 써도 요리는 안 된다. (같은 이름으로 두 번 `def` 하면 뒤엣것이 앞엣것을 **조용히 덮어쓴다.** 에러도 안 남)

괄호 안에 들어가는 게 서로 다르다

	괄호 안에 뭐가 들어가나
`def 함수(...)` **만들 때**	**빈 이름표** (`prompt`) — 실제 값 아님
`함수(...)` **부를 때**	**진짜 재료** (`chunks[0]`)

```
def embed_feedback(prompt):      # prompt = "누가 뭘 던져주면 안에서 prompt라 부를게"라는 약속
embed_feedback(chunks[0])        # chunks[0] = 실제로 던지는 재료
```

🐞 **오늘 한 실수:** 호출하려고 `def embed_feedback(chunks[0]):` 이라고 씀 → ① `def`라서 부르는 게 아니라 또 만든 것 ② 이름표 자리에 실제 값을 넣어서 문법 에러

🐞 **또 한 실수:** `return` 아래에 있던 `embed(response)`를 **지우지 않고 위로 옮김** → `return` 아래는 실행 안 된다는 건 맞게 알았지만, `embed`라는 함수는 **애초에 존재한 적이 없었다.** → 옮기면 `NameError: name 'embed' is not defined` (= "그런 이름 처음 듣는데?") → 예전에 메모처럼 적어둔 흔적이라 **지우는 게 정답**이었음

---

12. 확인용 3줄 — 한 줄씩 뜯어보기

```
vector = embed_feedback(chunks[0])
print(len(vector))
print(vector[:5])
```

1번 줄 — 읽는 순서는 **안쪽 → 바깥쪽**

```
vector  =  embed_feedback(  chunks[0]  )
  ③          ②                 ①
```

- ① `chunks[0]` — 조각 8개 중 0번을 꺼낸다 → 글자 덩어리
- ② `embed_feedback(...)` — 그 글자를 재료로 던져 함수 실행 → 숫자 3072개를 `return`
- ③ `vector =` — 돌아온 값에 이름표를 붙여 **받아둔다**

> ⚠️ **③이 없으면 결과가 사라진다.** `embed_feedback(chunks[0])`만 쓰면 실행은 되지만 값이 허공으로 날아감. **함수는 불러야 일하고, 결과는 받아야 남는다.**

`vector`는 내가 지은 이름. `v`든 뭐든 상관없다.

2번 줄 — `len()`으로 개수 세기

`print(len(chunks))` → `8` 과 완전히 같은 도구. 여기선 `3072`가 나온다.

3번 줄 — 슬라이싱으로 맛보기

`print(vector)` 하면 숫자 3072개가 다 쏟아져 화면이 도배된다. 앞 5개만 본다.

```
목록[1:]    # 1번부터 끝까지  ← 앞을 버릴 때  (chunking[1:] 에서 씀)
목록[:5]    # 처음부터 5개만  ← 뒤를 버릴 때
```

> 2·3번 줄은 **결과 확인용**이다. 없어도 프로그램은 돌아가지만, 없으면 됐는지 안 됐는지 알 수가 없다.

---

13. 🐞 실행 환경 에러 2개 (또 코드 문제가 아니었음)

① ImportError — 파이썬을 잘못 부름

```
ImportError: cannot import name 'genai' from 'google'
File "C:\Python314\python.exe"        ← 여기가 힌트
```

`python`이라고만 치면 **컴퓨터에 깔린 기본 파이썬(C:\Python314)** 이 잡힌다. 근데 `google-genai`는 **프로젝트 전용 파이썬(`.venv`)** 에 깔려 있다.

> **venv(가상환경)** = 프로젝트마다 따로 쓰는 파이썬. 프로젝트 A는 라이브러리 3개, B는 10개 — 섞이지 말라고 방을 나눠놓은 것.

해결 2가지

```
# 방법 1 — venv 파이썬을 콕 집어 부르기
Kids_Feedback\.venv\Scripts\python.exe Kids_Feedback\rag\kids.py

# 방법 2 — 활성화(activate). 터미널 창 하나당 한 번만
Kids_Feedback\.venv\Scripts\Activate.ps1
# → 프롬프트에 (.venv) 가 붙으면 성공. 그 뒤론 그냥 python 이라고만 쳐도 됨
```

② `.env`를 못 찾는 문제

```
Kids_Feedback/          ← 바깥. rag_docs 여기. 여기 서서 실행함
├── rag_docs/
└── Kids_Feedback/      ← 안쪽. .env 는 여기 있음
    └── rag/kids.py
```

`load_dotenv()`는 **지금 서 있는 곳부터 위로만** `.env`를 찾는다. 아래쪽 폴더에 있는 `.env`는 못 찾는다.

해결: 경로를 직접 알려준다 → `load_dotenv("Kids_Feedback/.env")` `open("rag_docs/...")`와 **같은 기준(바깥 폴더)** 이 되어 헷갈릴 게 없어진다.

---

14. 실행 명령어 최종본

```
cd C:\Users\USER\GITHUB\Kids_Feedback
$env:PYTHONIOENCODING="utf-8"
Kids_Feedback\.venv\Scripts\python.exe Kids_Feedback\rag\kids.py
```

⚠️ `cd Kids_Feedback`으로 **안쪽에 들어가면 안 된다.** 들어가면 경로가 3번 겹쳐서 에러. `ls` 쳐서 `rag_docs`, `study`, `todayplan.md` 보이면 맞는 위치.

---

15. 복습 체크리스트 (안 보고 떠올리기)

각 항목을 **30초 안에 내 말로** 설명할 수 있으면 통과.

- [ ] 임베딩이 뭐고 왜 하나? (한 문장)
- [ ] 왜 벡터 길이가 항상 3072로 고정돼야 하나?
- [ ] `def 함수(prompt)` 의 `prompt` 와 `함수(chunks[0])` 의 `chunks[0]`, 뭐가 다른가?
- [ ] `vector = 함수(...)` 에서 `vector =` 를 빼면 무슨 일이 생기나?
- [ ] `목록[1:]` 과 `목록[:5]` 의 차이는?
- [ ] `ImportError: cannot import name 'genai'` 를 봤다. 제일 먼저 의심할 것은?

---

16. 다음 차례

Day 9 ③ — 조각 8개 전부 벡터로 바꾸기

지금은 `chunks[0]` **하나만** 임베딩했다. 8개를 다 해야 검색이 된다. → 이미 쓸 줄 아는 그 패턴: **빈 통 만들기** → **`for`로 하나씩** → `append`

그다음 ④ 가 진짜 검색 (질문도 벡터로 → 8개 중 제일 가까운 것 찾기).

전체 흐름은 `study/Day9-14_구현지도.md` 참고.

---

17. 오늘 회고 (①②)

- 코드 자체(`split`, `for`, `strip`, `[1:]`)보다 **실행 환경**에서 더 많이 막혔다 → 경로·인코딩·venv는 개념만 알면 다시는 안 막히는 종류의 벽
- for문 뼈대를 처음 손으로 쳤다. ①에서 성공 → ③에서 한 번 더 쓴다
- **제일 많이 막힌 건 `def`(만들기)와 호출(부르기)의 구분.** 11번 섹션 반드시 재방문
- 에러 5개 전부 "코드가 틀려서"가 아니라 **"컴퓨터한테 위치/규칙/어느 파이썬인지를 안 알려줘서"** 였음

---

④ 검색 (Search) — 질문과 제일 가까운 조각 찾기

> 파일: `Kids_Feedback/rag/search.py` 결과: "소심한 애가 발표했는데..." 질문 → **위축형 칭찬 조각**을 뽑아냄 ✅

---

18. 완성한 코드

```
question = "소심한 애가 처음으로 발표했는데 뭐라고 해줘야 하죠?"
q_vector = embed_feedback(question)      # 질문도 숫자로 (for 밖 — 딱 1번)

scores = []                              # 빈 통 (for 밖)
for d in data:                           # 저장해둔 조각 8개를 하나씩
    scores.append(cos_sim(q_vector, d["vector"]))   # 질문 vs 조각 닮음 점수

best = scores.index(max(scores))         # 제일 높은 점수의 '자리 번호'
print(data[best]["text"])                # 그 자리 조각을 꺼내 보여주기
```

결과

```
질문: 위축형 아이가 어린 동생을 스스로 챙겨줬을때 뭐라고 말해주나요?
답: ...
```

질문이 "소심한 애"였는데 **"위축형"** 조각을 찾아왔다 = **글자가 안 겹쳐도 뜻으로 찾음.** ✅

---

19. 한 줄씩 뜯어보기

`q_vector = embed_feedback(question)` — 질문도 같은 자로 재야 한다

조각을 숫자로 바꿀 때 쓴 **바로 그 함수**로 질문도 숫자로 바꾼다. 👉 **비교하려면 둘 다 같은 방식으로 숫자화**해야 한다. (자를 다르게 쓰면 길이·기준이 안 맞아 비교 불가) ⚠️ **for문 밖에서 딱 1번.** 임베딩은 API 호출 = 돈·시간. 8번 돌 때마다 질문을 다시 숫자로 만들 이유가 없다.

`scores = []` → `for` → `append` — 또 그 패턴

①③에서 두 번 쓴 **빈 통 → for → append**. 이번엔 담는 게 조각이 아니라 **닮음 점수**. 🐞 오늘 실수: `scores = []`를 for문 **안**에 넣음 → 매 바퀴 통이 비워져서 점수 1개만 남음. **빈 통은 반드시 for 밖에.** (들여쓰기 = 소속)

`cos_sim(q_vector, d["vector"])` — 두 벡터가 얼마나 닮았나

코사인 유사도. **1에 가까울수록 닮음, 0에 가까울수록 남남.** 방향(각도)만 본다 → 글이 길든 짧든 공평하게 비교됨. (7번·②에서 "좌표로 거리 재기" 한 그거)

`best = scores.index(max(scores))` — 오늘의 핵심 한 줄

안쪽부터 읽는다:

```
best  =  scores.index(  max(scores)  )
  ③          ②              ①
```

- ① `max(scores)` — 점수 8개 중 **제일 큰 값**
- ② `scores.index(...)` — 그 값이 **몇 번째 자리**인지 (0~7)
- ③ `best =` — 그 자리 번호를 받아둔다

왜 자리 번호가 답이 되나: `scores`와 `data`는 **같은 순서**로 만들어졌다(for가 `data` 순서대로 돌며 `scores`에 담았으니까). 그래서 **"3번 점수가 1등"이면 "3번 조각"이 정답.** 이 짝이 맞는 게 핵심.

🐞 오늘 실수 2개:

- `best = print(...)` → **`print`는 화면에 보여주기만 하고 돌려주는 값은 `None`.** 값 담을 땐 print 벗기기.
- `data["text"]` → `data`는 리스트라 자리 **숫자**를 넣어야 함(`data[best]`). 글자 넣으면 `TypeError`.

---

20. 🐞 오늘 만난 함정 정리 (④)

#	틀린 코드	왜 틀렸나	고침
1	`for s in scores:{ }`	파이썬은 중괄호 안 씀	`:` + 들여쓰기
2	`for s in scores:`	`scores`는 아직 빈 통. 돌 대상은 `data`	`for d in data:`
3	`scores = []`가 for 안	매 바퀴 비워짐 → 1개만 남음	for **밖**으로
4	질문 벡터를 for 안	API 8번 = 돈 낭비	for **밖**에서 1번
5	`best = print(...)`	print는 `None` 돌려줌	print 벗기기
6	`data["text"]`	리스트엔 숫자 자리	`data[best]["text"]`

공통 교훈: ④는 새 문법이 0개였다. 전부 **"어디에 놓느냐(들여쓰기)"** 와 **"값을 담을 건가 보여줄 건가(print 구분)"** 싸움.

🗺️ 21. 전체 흐름 — 내 자료에 맞는 답이 나오는 과정 (RAG 큰 그림)

> **오늘 만든 조각들이 어떻게 이어져서 "우리만의 답"이 나오는가.** 이걸 이해하면 Day 8~9 전체가 한 줄로 꿰인다.

왜 RAG가 필요한가 — 문제부터

LLM(제미나이)은 **인터넷 일반 지식**만 안다. 우리 학원의 **"위축형 아이는 결과 말고 시도를 칭찬한다"** 같은 **우리만의 방식**은 **모른다.**

그냥 물어보면 → 그럴듯하지만 **우리 방식이 아닌** 일반론이 나온다. 그래서 답하기 **직전에** 우리 자료를 찾아서 **"이거 보고 답해"라고 손에 쥐여주는** 것 = **RAG.**

```
RAG = Retrieval(찾아서) + Augmented(보강한) + Generation(생성)
       └ 관련 자료 검색    └ 그걸 프롬프트에 넣어  └ LLM이 그걸 바탕으로 답
```

3단계로 나눠 보기

🔵 1단계 — 준비 (딱 한 번, 미리) → 오늘 ①②③

자료를 **검색 가능한 형태**로 미리 만들어 저장.

```
rag_docs/*.md (우리 자료)
   │ ① Chunking  (## 기준으로 자르기)
   ▼
조각 8개  ("질문1개 = 대답1개" 단위)
   │ ② Embedding (글 → 숫자 3072개)
   ▼
벡터 8개
```

```
   | ③ 저장 (json.dump)
   ▼
vectors.json   ← [{text, vector}, ... 8개]  ★한 번 만들어두면 끝★
```

💡 왜 파일로 저장? 임베딩은 **API 호출(돈·시간)**. 한 번 만들어두면 다음부턴 **읽기만**.

🟢 2단계 — 검색 (질문 올 때마다) → 오늘 ④

```
질문 "소심한 애가 발표를..."
   | 질문도 같은 함수로 Embedding
   ▼
q_vector (숫자 3072개)
   | vectors.json의 8개와 하나씩 cos_sim 비교
   ▼
점수 8개 → 제일 높은 것 = best
   ▼
제일 관련 있는 조각 1개  ("위축형 칭찬법 ...")
```

🟡 3단계 — 생성 (마지막) → **아직 안 함 (Day 10~11)**

찾은 조각을 **프롬프트에 끼워서** LLM한테 던진다.

```
prompt = f"""
아래 [참고자료]를 바탕으로 답해줘.

[참고자료]
{뽑은_조각}          ← 2단계에서 찾은 그거

[질문]
{question}
"""
답 = LLM(prompt)      # ← LLM이 '우리 자료'를 보고 답을 만든다
```

여기가 핵심: LLM은 원래 우리 자료를 몰랐지만, **프롬프트에 넣어주는 순간 그걸 보고 답한다.** → 그래서 **"내가 원하는 자료에 맞는 답"** 이 나온다.

한 문장 요약

> 미리 **잘라서(①) 숫자로 바꿔(②) 저장**해두고(③), 질문이 오면 **뜻으로 검색해서(④) 관련 자료를 찾아**, 그걸 **LLM 프롬프트에 넣어 답을 만든다(3단계).** LLM은 똑똑하지만 **우리 자료를 모른다** → RAG가 그 자료를 **답 직전에 손에 쥐여주는** 역할.

지금 어디까지 왔나

- ✅ ① Chunking / ✅ ② Embedding / ✅ ③ 저장 / ✅ ④ 검색
- ⬜ 3단계(찾은 조각 → LLM 프롬프트 → 최종 답) — **Day 10~11**
- ⬜ Streamlit로 화면 붙이기 — 그다음

지금 우리는 **"관련 자료를 정확히 찾아오는" 데까지** 완성했다. 마지막 한 조각(찾은 걸 LLM에 먹여 답 만들기)만 남았고, 그게 RAG의 결승선이다.

---

22. 오늘 회고 (④ + 전체)

- ④는 새 개념 0개. **위치(들여쓰기)와 print 구분**만으로 이뤄진 세션 → 기초 문법이 손에 붙는 중
- `best = scores.index(max(scores))` 를 **안쪽→바깥쪽**으로 읽는 습관이 생김
- RAG 4조각(chunk→embed→저장→검색)이 **하나로 연결되는 그림**을 이제 말로 설명할 수 있음
- 다음: 찾은 조각을 **LLM에 먹여 최종 답 만들기**(3단계) = RAG 결승선